

王林海

电话: 17307179854 邮箱: 17307179854@163.com

年龄: 22 岁 性别: 男 工作年限: 应届生 求职意向: AI应用全栈开发工程师 政治面貌: 群众



专业技能

- 修读 Web 开发技术综合实训 (Vue)、Web 前端框架技术等课程, 具备 Vue 框架基础, 可快速上手主流前端技术栈。
- 熟悉前后端分离架构与 RESTful 接口设计规范, 能独立完成接口联调与跨端数据流处理, 了解 SSE 流式推送与移动端 H5 适配。
- 熟练掌握 Python, 完成两个项目的后端全链路开发, 涵盖接口设计、业务逻辑、异步任务与异常处理。
- 掌握 LangGraph 多智能体编排、ReAct 推理模式、MCP 工具集成与 Function Calling, 完成 AI 应用的核心链路开发。
- 熟悉 Redis 缓存、PostgreSQL 数据库、RabbitMQ 消息队列, 具备高并发场景下的异步削峰与数据一致性处理经验。
- 修读 Java 面向对象程序设计、Web 开发技术 (Java)、SpringBoot 企业级开发、SpringCloud 微服务架构等课程, 具备 Java 语言与微服务理论基础。
- 掌握 RAG 检索增强生成全链路: 文本向量化 (bge-m3 1024 维)、稠密向量检索、BM25 稀疏检索、RRF 融合、BGE-reranker / gte-rerank 重排序, 检索 Top-K 准确率做到 95% 以上。
- 熟悉 Chroma / Milvus 向量数据库与 MinIO 对象存储, 具备非结构化数据的采集、清洗、入库与检索的完整实践经验。
- 熟悉 Prompt 工程与大模型 API 调用、成本控制与效果调优, 理解 token 消耗与推理降本和优化路径。
- 熟悉 Git / GitHub 版本管理与分支协作、Docker 容器化。
- 熟悉 GitHub Actions CI/CD, 搭建三阶段流水线与 14 项测试门禁, 实现版本可追溯、发布可回滚。
- 熟练使用 AI Coding 工具 (Cursor / Copilot 类) 辅助开发, 理解生成代码的质量把关与验证方法。

教育背景

广州工商学院

软件工程 (本科)

2023.09 - 2027.06

主修课程: 数据结构与算法、操作系统、计算机网络、数据库原理与应用、软件工程、Python 基础与办公自动化应用、面向对象程序设计 (Java)、Web 开发技术 (Java)、SpringBoot 企业级开发、SpringCloud 微服务架构、Web 开发技术综合实训 (Vue)、数据库开发技术 (openGauss)、Linux 操作系统、软件质量保证与测试

项目经历

基于LangGraph的智能客服Agent平台

全栈独立开发

2026.05 - 至今

项目背景: 企业客服场景痛点集中于用户咨询意图复杂, 高峰排队超 30 分钟; 知识问答依赖专家, FAQ 直达率低; 技术问题排查周期长, 人工响应慢; 运营成本高, 7×24 小时值班压力大。

项目目标: 构建覆盖用户接入→意图理解→知识检索→自主推理→人工兜底→服务闭环全链路的企业智能客服 Agent 平台, 实现数字员工式智能问答与知识库服务。

核心工作:

- 基于业务方响应慢、人力成本高的核心诉求, 调研并对比主流大模型应用框架, 最终选定 LangGraph 作为编排底座, 设计 4 智能体 A2A 协作架构与 8 节点状态机 workflow, 独立完成后端开发, 将 FAQ 首次响应压至 1 秒以内。
- 基于知识检索准确率不足导致回答质量差的问题, 评估稠密向量、BM25 稀疏检索与 RRF 融合方案的技术取舍, 选定混合检索加 BGE-reranker / gte-rerank 重排序方案, 完成全链路开发与参数调优, 将检索 Top-K 准确率提升至 95% 以上。
- 基于 AI 需要操作外部业务系统的需求, 通过 MCP 协议完成 PostgreSQL、飞书、GitHub 外部工具的接口对接与集成, 让 AI 能力与既有业务系统打通。
- 基于多轮对话上下文易断裂的问题, 设计并实现 Redis 短期记忆、PostgreSQL 长期记忆加上下文注入的三层记忆架构, 配合 RabbitMQ 异步写入削峰, 保障高峰期服务稳定不丢数据。
- 独立实现 React 前端, 完成多轮对话交互、流式响应展示与会话管理, 建立前后端接口规范, 保证交互体验流畅。

- 负责部署与运行保障，完成 Docker 容器化与 Kubernetes 部署配置，搭建 GitHub Actions CI/CD 流水线与日志监控机制，实现版本可追溯、发布可回滚。

项目成果：

- FAQ 首次响应时间降至 1 秒以内，技术问题排查平均 2-5 秒
- LLM 调用减少 60% 至 80%，推理成本显著降低
- 高频咨询响应速度提升约 45%，工单处理效率提升约 3 倍
- 全栈独立开发并开源 (github.com/addhai)，含完整 CI/CD 与容器化部署配置

基于LangGraph的智能招聘中台

全栈独立开发

2026.07 - 至今

项目背景： 传统招聘流程高度依赖人工，HR 单份简历综合处理（筛选→匹配→安排→评估）耗时约 3 小时；简历匹配无量化标准，优质人才易遗漏；全链路碎片化，招聘效率低下。

项目目标： 从零搭建招聘全链路 AI 自动化中台，实现「简历解析→智能匹配分流→自动问询筛选→面试智能排期→多维度综合评审→录用决策 / 人才池沉淀」完整业务闭环。

核心工作：

- 基于 HR 简历筛选耗时冗长的痛点，调研并设计 LangGraph 多节点 workflows 方案（简历解析、人岗匹配、面试安排、综合评估），独立完成后端开发，结合 SSE 实时推送提升交互体验，将简历处理从 3 小时压缩至 30 分钟。
- 基于非结构化简历数据的检索需求，完成 Chroma / Milvus 与 MinIO 数据方案的设计与集成，使用 bge-m3（1024 维）完成向量化，构建语义级人岗匹配能力。
- 集成 PaddleOCR / Tesseract 完成扫描版简历的文字识别与结构化提取，补齐系统的数据接入能力边界。
- 独立实现 React 前端，完成流程看板、实时进度推送与结果展示。
- 负责交付质量建设，搭建 GitHub Actions 三阶段 CI 流水线，编写 14 项后端测试用例并设为合并门禁，实现代码变更自动验证。
- 独立撰写需求说明、接口文档与部署手册，保障项目可复现、可交接。

项目成果：

- HR 单份简历综合处理时长从 3 小时压缩至 30 分钟
- 多维度评审耗时降至原方案的二分之一（节省 50% 时间）
- 全流程自动化替代大量人工重复操作，显著提升人才池覆盖率
- 全栈独立开发并开源 (github.com/addhai)

自我评价

计算机专业本科生，专注 AI 应用全栈开发，独立完成两个已开源的 LLM 应用产品，涵盖 React 前端、FastAPI 后端、向量数据库、Agent 编排、容器化部署完整技术栈，两个项目均从需求分析一路做到生产级交付。熟练使用 Python，具备 Java 课程基础。习惯使用 AI Coding 工具提升开发效率，对 LangGraph、RAG、MCP 等新技术保持持续跟进与落地实践，所有技术栈均靠课外自学掌握。有完整的技术文档产出，具备项目交付与推广所需的表达能力。